FP Design Document

Max Harmsen (2335778) Mika Kalshoven (2285061)

Utrecht University, Utrecht, The Netherlands
October 2026

1 Introduction and Rules

This document will explain how we will implement the game Pac-Man for our final assignment of Functional Programming. The goal with Pac-Man is to eat as much of the little dots on the screen as you can, while not being captured by one of the 4 colorful ghosts floating around the field. The dots are spread across a maze-like structure where Pac-Man (a yellow circle with a mouth) should maneuver its way through. The game ends when you get caught by a ghost 3 times. There are also 4 large dots randomly placed throughout the maze called power pellets. When these power pellets are eaten by Pac-Man, the ghosts turn into helpless objects that can be eaten by Pac-Man and provide extra points. The overall goal of the game is to get as many points as possible.

2 Design

2.1 Data structures

2.1.1 General

Position describes where on the board (see Section 2.1.3) an object is located

```
type Position = (Int, Int)
```

Direction describes where either Pac-Man or a Ghost (see Section 2.1.4) is going or wants to go.

```
data Direction = North | East | West | South
```

2.1.2 Game state

GameStatus describes in what state the game currently is.

```
data GameStatus = Ongoing | Paused | NStarted | Lost | Won
```

The **GameState** contains everything there is to know about the game, including the scores, the remaining lives of Pac-Man and the position of all objects.

```
data GameState = GameState {
                                board           :: Board,
                                pacMan          :: PacMan,
                                ghost           :: [Ghost],
                                dots            :: [Dot],
                                score           :: Int,
                                lives           :: Int,
                                status          :: GameStatus,
                                timerOngoing    :: Float

}
```

2.1.3 Board

The board describes the playing field of the player. It consists of a list of walls. The length of a wall is always measured left to right. The **Board** has a dimension (since the board is always a square only one number is needed) and a list of walls.

```
data Wall   = Wall {
                        length          :: Int,
                        startPosition   :: Position
}
data Board  = Board {
                        walls       :: [Wall],
                        boardDimensions :: Int
}
```

2.1.4 Objects

Pac-Man has a current position, a direction it is currently going and a direction that points to the user's last input. Pac-Man will change to the wanted direction as fast as possible, but since it cannot go through walls, it happens that directionGoing and directionWant are not the same. In that case, Just Direction describes the input. When the user doesn't click any button and the last directionWant is the same as directionGoing, directionWant becomes Nothing.

```
data PacMan = PacMan {
                            position        :: Position,
                            directionGoing  :: Direction,
                            directionWant   :: Maybe Direction
}
```

A **Ghost** can be one of four types (see Section 2.2). It also is in one of four states, where Afraid Float describes how long it takes for the ghost to be afraid, and Return describes that the ghost was eaten and has to return to the home base to continue.

```
data GhostType = Blinky | Pinky | Inky | Clyde
data GhostStatus = Afraid Float | Normal | Return | Waiting
data Ghost     = Ghost {
                            name            :: GhostType,
                            position        :: Position,
                            status          :: GhostStatus,
                            directionGoing  :: Direction
}
```

Dots can either be small or big, where small just grants Pac-Man points when eaten, but big makes the ghosts afraid for a certain amount of time (see Section 2.2.5 for detailed explanation about afraid ghosts).

```
data DotType = Small | Big
data Dot = Dot {
                    eaten       :: Bool,
                    dotType      :: DotType,
                    dotPosition :: Position
}
```

2.2 Computer Adversary

Each game will have four computer players, also known as the four ghosts. Each ghost has a slightly different algorithm to make it as difficult as possible for the player to score points. The ghosts are called Blinky, Pinky, Inky and Clyde (see Section 2.1.4 for the data structure that belongs to the ghosts). Ghosts will never make a 180 degree turn. At crossings, they will always choose the direction that brings the ghost closest to its target (in their normal state). What the target of the ghost is, is defined below.

2.2.1 Blinky

Blinky is the most aggressive ghost and is often referred to as Pac-Man's greatest enemy. His algorithm consists of following the current position of Pac-Man and trying to catch him.

2.2.2 Pinky

Pinky, also known as Speedy, is the pink ghost and is the only Female ghost in the game. Her objective is to ambush Pac-Man. To accomplish this, her algorithm will aim to get 4 tiles in front of Pac-Man, based on his current direction.

2.2.3 Inky

Inky is the most complicated of the four main ghosts. His movement is determined by a 2 vector analysis. His direction is based on both the position of Pac-Man and Blinky, making it seem random when it in fact is very much according to a logic. The first point that is taken for the algorithm is two tiles in front of Pac-Man in the direction he is currently facing. Then, calculate the vector between that point and **Blinky**. The vector resulting from that should be doubled. The doubled vector should be added to **Blinky's** current position and that will be the target position of **Inky**

2.2.4 Clyde

Clyde is the most "chaotic" of the ghosts. His movement is determined by the distance to Pac-Man. When the distance between him and Pac-Man is more than 8 tiles, he will chase Pac-Man like **Blinky**. However, if the distance between him and Pac-Man is less than 8 tiles, he will scatter to the bottom left corner. This makes **Clyde** hard to predict.

2.2.5 Afraid

As soon as a ghost enters his afraid state, he turns into an unpredictable object which is slower than a normal ghost in order for Pac-Man to catch him. Every crossroad, the ghost randomly chooses a direction (except from turning back as the rule at the beginning stated). There is no further logic behind the movement of a scared ghost.

2.3 Interface

2.3.1 Layout

The player will interact with our game through a visual interface which shows a maze like structure. The maze will be made out of a pattern of walls placed on a grid. Every box that isn't a wall, is a path for the objects to move around in (see Section 2.1.4 for a detailed explanation about the objects). Each grid position that isn't the starting position of Pac-Man or the cage of the ghosts and is not a wall will either be a big dot or a small dot (see Section 2.1.4 for detailed explanation about the dots).

2.3.2 From grid to screen

The grid will be divided into a set of squares, each of the same size. Each square will count as one step. Since each square has a fixed screen size, the grid can be converted to screen coordinates to eventually display the board and objects.

2.3.3 Drawing

To draw the items, we will make use of the *gloss* library. To do this, we will use the functions shown below that will later be defined more precisely. Every function but drawGame will be used to draw one of the data structures defined in Section 2.1. drawGame will then use all the functions to draw the entire board with corresponding objects.

```
drawGame   :: GameState -> Picture
drawBoard  :: Board -> Picture
drawDot    :: Dot -> Picture
drawPacMan :: PacMan -> Picture
drawGhost  :: Ghost -> Picture
```

2.3.4 Looks

- Pac-Man will be displayed as a yellow circle with a small black dot which represents the eye and a triangle that represents its mouth in the direction Pac-Man is moving.
- Walls are represented as blue rectangles.
- Small dots and power pellets are represented as small and large gray circles.
- The score will be a white text represented below the playing field
- Ghosts each have their own color (Blinky red, Pinky pink, Inky cyan and Clyde orange). The rest of the ghost will be identical, the shape and the eyes. When returning, the ghost will only be represented by its eyes. When afraid, the ghosts turn into a flickering blue to white state.

2.4 Implementation of minimum requirements

2.4.1 Player

The player will be able to control the "Pac-Man". This is a small, simple yellow circle that can move around in the map with the controls WASD, for up-, left-, down-, and right movement, respectively.

2.4.2 Enemies.

We will follow the original rules of the game, meaning that there are 4 ghosts: Blinky, Pinky, Inky & Clyde, each with their own behaviour and style (see Section 2.2 for detailed explanation).

2.4.3 Randomness

The randomness in our game will be provided by randomly placing the "power pellets" in the map. Power pellets are the large circles that Pac-Man can eat to start eating Ghosts. On top of that, the afraid ghosts themselves also choose a random direction at every cross point (See Section 2.2 for a more detailed explanation).

2.4.4 Animation

The animation will come from the part when Pac-Man eats a ghost or eats a power pellet. When Pac-Man eats a power pellet, all the ghosts will start to flash blue and white. And when one is eaten, the ghost will only have his eyes left, and will scatter back to his spawn.

2.4.5 Pause

The pause feature will be quite simple, when the player presses ESC, the game will pause, meaning that no input from the keyboard will register to Pac-Man's movement, and the game state will freeze until the game continues. The game can be continued by pressing the space bar.

2.4.6 Interaction with the file system

The player's current **GameState** will be stored in a text file (.txt) on pause. Later, the player can then resume from that file to continue its play.

2.5 Implementation of the optional requirements

2.5.1 Levels

The game can be played with multiple maze patterns.

2.5.2 Different enemies

See the explanation about Blinky, Pinky, Inky, and Clyde in the enemies section.

2.5.3 Custom levels

Player will be able to play their own custom levels that they themselves have created. The way it will be created is by a simple grid where the player can click boxes with their mouse, which will then become walls. The player can also place food or power pellets. This map will then be saved in a JSON-file.

2.5.4 Mouse input

The mouse input in our game will be a little different, because while playing the game, the player won't need a mouse. It would not make sense to implement a feature where the Pac-Man is controlled by a mouse. However, what we can do is use the mouse for another function. See 2.5.3 for more information.

2.5.5 Multi-player

For the multiplayer feature, we will implement a second playable character in the game that can be controlled by the keyboard arrows (up, down, left, right). The player will have the option to play as the antagonist, meaning a ghost, or as a second Pac-Man (Mrs Pac-Man). This will also change the behavior of the non-playable-characters (NPC) ghosts and the level. In multiplayer mode, the map will be larger, and there will be twice as many ghosts, two "squads" for the two protagonists.

2.5.6 Use JSON to save the full game state

No further explanation needed here, we will implement a JSON save in the pause menu. This is different from the default save that happens immediately on pause (see Interaction with the file system in Section 2.4)